A From-Scratch Deep Learning Engine with Distributed Training via Custom RPC

Fahd Makdad
2nd Year - Computer Science
Faculté des Sciences et Techniques Settat
Settat, Morocco
makdad.fahd8371.fsts@uhp.ac.ma

***Abstract*—This project is a deep learning framework built entirely from scratch in Python and NumPy, without relying on existing libraries such as PyTorch or TensorFlow. Its first component is an n-dimensional autograd engine, inspired by Andrej Karpathy's scalar-valued micrograd, extended to support multi-dimensional tensor operations and matrix-based neural networks. The engine implements reverse-mode automatic differentiation, a multilayer perceptron, and the Adam optimizer, which I benchmark against SGD, SGD with momentum, and RMSProp, alongside a comparison of ReLU, Tanh, and Sigmoid activation functions. To validate correctness, I compare the engine directly against PyTorch on MNIST, using identical architectures and initial weights across 10 random seeds: the engine reaches 97.62% ± 0.13% accuracy, close to PyTorch's 97.45% ± 0.28%. The second component extends the engine into a distributed training system, using a custom Remote Procedure Call (RPC) layer built over raw TCP sockets, without external libraries such as gRPC. I identify and resolve a significant network bottleneck in this system, bringing epoch time down from around 6 minutes to 8.14 seconds through batched tensor transmission, mixed-precision serialization, and TCP-level tuning, with no loss in accuracy. Finally, I benchmark synchronous versus asynchronous multi-worker training, demonstrating that while asynchronous updates reduce wall-clock time, they suffer from gradient staleness, leading to a noticeable drop in final model accuracy.**

I. Introduction & Motivation

After completing my first year of Computer Science, in which coursework was limited to algorithms in C, I decided to pursue a project that would go beyond what the curriculum covered — something substantial enough to genuinely deepen my understanding of machine learning, and valuable enough to support future academic applications. Modern machine learning development relies heavily on frameworks such as PyTorch and TensorFlow, which abstract away the underlying mechanics of automatic differentiation and distributed training. While effective for building applications quickly, this abstraction can leave practitioners with a limited understanding of what actually happens beneath the high-level API. This project was motivated by a desire to close that gap: to understand, by building it myself, exactly how gradients are computed, how neural networks are trained, and how training can be distributed across multiple machines — rather than relying on existing solutions such as PyTorch's autograd or pre-built communication libraries such as gRPC. The project is structured in two parts. The first is a deep learning engine, implemented entirely from scratch in Python and NumPy, covering automatic differentiation, a multilayer perceptron, and the Adam optimizer. The second extends this engine into a distributed training system, built on a custom Remote Procedure Call (RPC) layer implemented directly over raw TCP sockets, without relying on any pre-built RPC framework. The remainder of this document describes the design, implementation, and empirical validation of both components, along with a detailed account of a significant performance investigation undertaken during the development of the distributed system.

Source code available at: https://github.com/makdadfahd/dist-ml-framework

II. The Core Engine

The engine is built around a custom `Tensor` class, where each tensor stores three things: its data, its gradient, and references to the tensors that produced it (its "children"). For an operation such as $C = A + B$, the resulting tensor C stores pointers to both A and B as its children. As operations are chained together, each new tensor records the tensors it was built from, forming a computational graph.

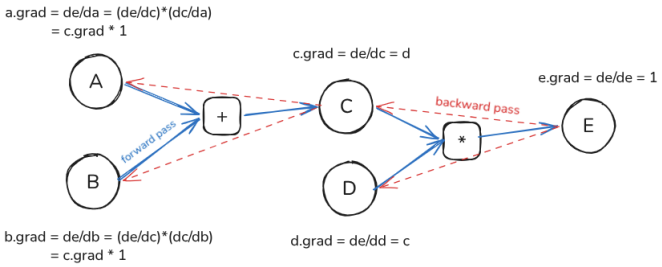


Fig. 1. The computational graph for the expression $E = (A + B) \cdot D$. Solid arrows show the forward pass; dashed arrows show the backward pass, with each node's gradient computed via the chain rule, as described below.

To compute gradients, the engine performs reverse-mode automatic differentiation, applying the chain rule from calculus starting at the final output and working backward toward the inputs. Given a loss L, the gradient $\partial L/\partial L$ is initialized to 1, while all other gradients start at 0. For any operation connecting two tensors, the chain rule allows the gradient to be propagated to each input by multiplying the gradient of the output by the local derivative of the operation – for example, for $e = c \cdot d$, $\partial L/\partial c = (\partial L/\partial e)(\partial e/\partial c)$. By repeating this at

every node, gradients flow backward through the entire graph until every parameter has received its gradient.

A tensor can be used as input to more than one operation, meaning its gradient depends on contributions arriving from multiple downstream paths. To ensure every contribution has arrived before a tensor's gradient is finalized, the engine performs a topological sort of the graph before calling `.backward()`, guaranteeing that a node is only processed after every node that depends on it has already propagated its gradient back. Without this ordering, a tensor's gradient could be read before all of its contributions had arrived, producing an incomplete result.

A. Broadcasting and Gradient Reduction

Extending this system from scalars to n-dimensional tensors introduced a significant challenge: broadcasting. When a smaller tensor is combined with a larger one – for instance, adding a bias vector of shape $(3,)$ to a batch of inputs of shape $(4, 3)$ – NumPy implicitly stretches the smaller tensor across the batch dimension without duplicating it in memory. During the backward pass, this creates a shape mismatch: the incoming gradient has the shape of the larger tensor, while the smaller tensor's own gradient must match its original, smaller shape. Since the smaller tensor was effectively reused across every row of the batch, its gradient must sum the contributions from all of those rows.

Concretely, if a bias vector $b = [1, 2, 3]$ is added to a batch of two inputs, and the gradient arriving at the output is

$$\frac{\partial L}{\partial C} = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \end{bmatrix} \tag{1}$$

then b.grad is obtained by summing down the batch dimension:

$$b.\text{grad} = [0.5,\ 0.7,\ 0.9] \tag{2}$$

restoring the original shape. Correctly implementing this "unbroadcasting" step for both addition and matrix multiplication was the most challenging part of building the engine.

B. Optimizer and Activation Benchmarks

On top of this autograd system, the engine implements a multilayer perceptron and the Adam optimizer. To evaluate design choices, I benchmarked Adam against SGD, SGD with momentum, and RMSProp on MNIST (Table I). Adam reached 97.26% test accuracy using its default learning rate, with no tuning required. RMSProp performed marginally better but required manual learning rate adjustment to avoid overshooting. SGD and SGD with momentum both required extensive tuning and underperformed Adam, with SGD with momentum reaching only 90.39% accuracy.

Using Adam, I additionally compared activation functions, (Table II), finding that ReLU, Tanh, and Sigmoid performed comparably, with ReLU achieving the highest accuracy.

The results in (Table I) and (Table II) are obtained from a single run and provide an initial comparison of the optimizers and activation functions using NumPy seed 42.

TABLE I
OPTIMIZER COMPARISON ON MNIST

Optimizer	Test Accuracy	Final Loss
Adam	97.26%	0.0475
SGD	97.07%	0.0801
SGD with Momentum	90.39%	0.3244
RMSProp	97.90%	0.0234

TABLE II
ACTIVATION FUNCTION COMPARISON ON MNIST

Activation	Test Accuracy	Final Loss
ReLU	97.84%	0.0193
Tanh	97.72%	0.0175
Sigmoid	97.57%	0.0494

C. Validation Against PyTorch

To validate the engine's correctness, I compared it directly against PyTorch on MNIST, using identical architectures, learning rates, hyperparameters and initial weights, across 10 random seeds. To ensure a controlled comparison and eliminate initialization bias, I implemented a Python function that copies the initial weights from the PyTorch model to my custom model, guaranteeing both models start from an identical state. My model achieved $97.62\% \pm 0.13\%$ test accuracy, compared to PyTorch's $97.45\% \pm 0.28\%$, with nearly overlapping loss trajectories throughout training (Figure 2), indicating the two implementations achieved comparable test accuracy across the evaluated random seeds.

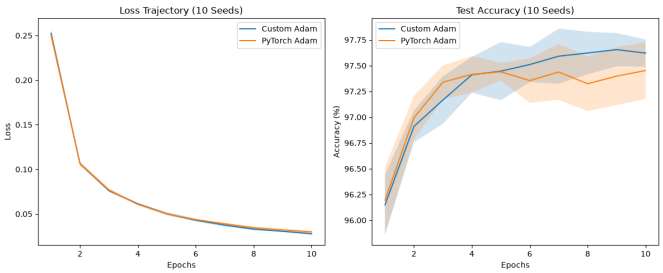


Fig. 2. Loss trajectory and test accuracy, custom model vs. PyTorch, mean $\pm$ std across 10 seeds.

D. Speed Comparison with PyTorch

In addition to matching PyTorch's accuracy, I compared the two engines' training speed. Across the 10-seed benchmark, my engine averaged 5.87 seconds per epoch (excluding two anomalous epochs caused by a transient system memory issue unrelated to the engine itself), compared to PyTorch's 2.28 seconds per epoch (Table III).

TABLE III
AVERAGE EPOCH TIME, CUSTOM ENGINE VS. PYTORCH (10 SEEDS)

Implementation	Avg. Epoch Time
Custom Engine	5.87 s
PyTorch	2.28 s

This gap is expected and does not reflect a flaw in the underlying mathematics, which was previously shown to match PyTorch within noise. PyTorch's speed advantage on CPU comes primarily from its C++ backend and its use of highly optimized linear algebra libraries, such as Intel MKL or OpenBLAS, which perform operations like matrix multiplication using vectorized, low-level code. My engine, by contrast, executes through NumPy directly from Python, suffering the overhead of the Python interpreter at each operation. This result reinforces that the value of building the engine from scratch lies in understanding and correctness, rather than in matching the raw performance of a mature, compiled framework.

III. Distributed Training System

The second part of this project extends the engine into a distributed training system built on a custom Remote Procedure Call (RPC) layer, implemented entirely from raw TCP sockets.

A. Parameter Server Architecture

The server holds the master copy of the model's parameters and an Adam optimizer. A worker connects to the server once and, for each batch of training data, calls `get_weights()` to obtain the current parameters, performs a forward pass, computes the loss, and calls `.backward()` to obtain gradients locally. These gradients are sent to the server through `push_grads()`, which sets them as the gradient of each corresponding server-side parameter, performs an Adam optimizer step to update the weights, and clears the gradients in preparation for the next round. This cycle repeats for every batch until training completes (Figure 3).

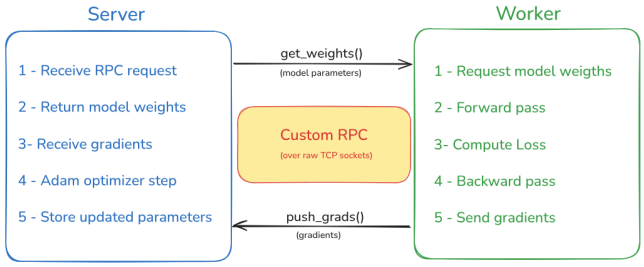


Fig. 3. The parameter server architecture: a worker repeatedly pulls the current weights, computes gradients locally, and pushes them back to the server, which applies an Adam update.

B. Avoiding Pickle for Serialization

I deliberately avoided Python's built-in `pickle` module for serializing data sent over the network. Pickle is capable of executing arbitrary code upon deserialization, meaning a malicious or corrupted payload received over a network could execute unintended code on the receiving machine without any visible warning. Although this project runs on a local network, I implemented my own serialization protocol to keep the system secure and generalizable to untrusted network environments.

C. Building a Custom RPC Layer

Consistent with the project's goal of building every component from first principles, I implemented my own RPC layer from raw sockets rather than relying on a pre-built library such as gRPC. This choice was made both to deepen my understanding of how data is actually transmitted between machines, and to retain full control over the protocol's design. The server maintains a dictionary mapping function names (e.g., `"get_weights"`, `"push_grads"`) to their corresponding methods. A `dispatch` method checks whether a requested function exists in this registry: if it does, the function is called with the provided arguments and its result returned; if not, an error response is returned instead. Workers invoke remote functions through a `call` method, which sends a request containing the function name and arguments. Since tensor arguments cannot be directly serialized, each tensor is replaced in the request with a placeholder marker, and the actual tensor data is transmitted separately, immediately following the request, in the same order as its corresponding placeholder. On the server side, a `serve_request` method receives this structured request, reconstructs any tensor arguments from the accompanying data, and passes the complete set of arguments to `dispatch`, returning its result back to the worker.

All messages are transmitted using a length-prefixed protocol: a message is serialized to JSON and encoded to bytes, its byte length is sent first as a fixed 4-byte integer, and the message itself follows, allowing the receiver to read exactly the expected number of bytes before decoding and parsing it. To transmit tensors, multiple tensors are sent together as a single batch rather than individually: for each tensor, the underlying data is first converted to half precision (float16) to reduce its size, then serialized to raw bytes. A combined list of metadata (dtype, shape, and byte length for each tensor) is sent first as a single message, followed by the concatenated raw bytes of all tensors in one transmission. On the receiving end, the metadata is read first, and each tensor is reconstructed in order from its corresponding bytes using `construct_tensor`, which restores the original shape and converts the data back to float32 for subsequent computation.

D. The Network Bottleneck

While the distributed system trained correctly, achieving the same accuracy as the single-machine version, it was significantly slower: a single epoch took approximately 6 minutes, compared to 5.87 seconds for local training. I first considered reducing synchronization frequency, either by exchanging weights and gradients only once every 10 batches or by allowing the worker to perform local optimizer steps between synchronizations; both approaches reduced communication but came at a measurable cost in final accuracy (93.69% and 93.88%, respectively), confirming that less frequent synchronization introduces gradient staleness. I therefore returned to synchronizing on every batch, which preserved full accuracy, and instead investigated the source of the per-call overhead itself.

E. Optimizing the Communication Layer

I identified three sources of overhead in the original implementation and addressed them in sequence, measuring epoch time after each change (Table IV).

First, tensors were originally sent one at a time, each suffering the overhead of a separate network round-trip. Batching all tensors into a single combined transmission, sent together as one block, reduced epoch time from approximately 6 minutes to 90 seconds.

Second, I compressed tensor data from 32-bit to 16-bit floating-point precision before transmission, converting it back to 32-bit on arrival. This halved the volume of data transmitted and reduced epoch time further, to 85 seconds – a smaller improvement than batching, since most of the overhead had already been eliminated.

Third, and most significantly, I discovered that `TCP_NODELAY` was disabled by default on my system. By default, TCP applies Nagle's algorithm, which delays sending small packets in order to batch them together for network efficiency – an optimization well suited to bulk data transfer, but harmful to latency-sensitive, frequent round-trip communication such as my get-weights/push-gradients cycle. Enabling `TCP_NODELAY` removed this artificial delay and reduced epoch time to 8.14 seconds, with no loss in accuracy.

TABLE IV
EPOCH TIME ACROSS OPTIMIZATION STAGES

Stage	Epoch Time
Per-batch sync (baseline)	∼6 min
Batched tensor transmission	90 s
Float16 compression	85 s
TCP_NODELAY enabled	8.14 s

This final configuration brings distributed training within a factor of approximately two of local single-machine training time (5.87 s per epoch), while preserving the full accuracy achieved by per-batch synchronization.

All optimization measurements above were obtained using a single worker. The following experiment extends the setup to two workers to investigate the effect of synchronization strategies on training time and accuracy.

F. Synchronous vs. Asynchronous Training

To evaluate the trade-offs of distributed coordination, I benchmarked synchronous and asynchronous training using two workers over 10 epochs. In the asynchronous configuration, each worker independently pulls the current weights, trains locally, and pushes its gradients back without waiting for the other. While this removes synchronization barriers, it introduces a critical trade-off.

As shown in Table V, asynchronous training completed slightly faster in 946.33 seconds with a final accuracy of 87.67%. In contrast, synchronous training took 980.49 seconds but achieved a significantly higher accuracy of 96.96%.

The asynchronous mode provided only a marginal speedup of approximately 3.5%, but resulted in a 9.29 percentage-point decrease in final accuracy. I attribute this degradation to gradient staleness because updates are applied without coordination, a worker's gradient is frequently computed against weights that have already been modified by the other worker. Consequently, outdated gradients are applied to the model, degrading the quality of each update. These results suggest that synchronous averaging provides more stable convergence in my architecture, while asynchronous updates offer only a marginal reduction in training time.

TABLE V
PERFORMANCE COMPARISON: SYNCHRONOUS VS. ASYNCHRONOUS TRAINING

Training Mode	Total Training Time	Average Final Accuracy
Synchronous (2 Workers)	980.49 s	96.96%
Asynchronous (2 Workers)	946.33 s	87.67%

Accuracy represents the average final test accuracy of the two workers.

IV. DISCUSSION AND FUTURE WORK

Beyond its technical results, this project represented a significant personal milestone. Having completed only a first-year curriculum centered on algorithms in C, taking on a project requiring an entirely unfamiliar domain – machine learning, linear algebra, and later, distributed systems – was a deliberate step outside my comfort zone. The most valuable outcome of this project was not any single technical result, but the confidence gained in approaching unfamiliar and difficult problems: the understanding that no project is too difficult to figure out, given enough persistence and a willingness to learn whatever is required along the way.

The results presented in this paper show that my engine's mathematical correctness closely matches PyTorch's, both in accuracy and in the behavior of training dynamics, across repeated trials. This project was not intended to match PyTorch's raw performance, which benefits from years of low-level optimization, but to build a correct and complete understanding of what such frameworks do internally. In this respect, the project succeeded: every core mechanism, from automatic differentiation to distributed gradient synchronization, was implemented, understood, and verified independently.

The distributed component of this project introduced an entirely new area of computer science for me: how independent machines communicate, coordinate, and share work over a network. Designing and debugging this system – including diagnosing and resolving the network bottleneck described in Section III – provided a concrete, practical understanding of distributed systems that is difficult to obtain from coursework alone.

A natural direction for future work is narrowing the execution-time gap with PyTorch. The current engine executes entirely through NumPy from Python, incurring interpreter overhead at every operation. A planned follow-on project, a custom compiler translating the engine's computational graph into optimized C++ and, eventually, CUDA code, aims to bypass this overhead for performance-critical operations.

This represents a substantially larger undertaking than the present work, requiring the construction of an intermediate representation, a code generator, and a GPU backend, but offers a concrete path toward closing the performance gap documented in Section II-D while preserving the correctness already established.

ACKNOWLEDGMENTS

I would like to thank Professor Said El Kafhali for his time, guidance, and encouragement throughout the development of this project.

REFERENCES

[1] A. Karpathy, *micrograd*, GitHub repository, https://github.com/karpathy/micrograd.
[2] M. Nielsen, *Neural Networks and Deep Learning*, Determination Press, 2015.
[3] A. Ng, *Machine Learning Specialization*, Coursera, DeepLearning.AI and Stanford University.
[4] Stanford University, *CS231n: Convolutional Neural Networks for Visual Recognition*.